



Tutorial

Machine Learning Method in Economic Signal Prediction

FUCHIEN TSOU¹

¹Graduation Institute of Communication Engineering, National Taiwan University, Taipei, Taiwan.

Keywords: keyword1, keyword2, keyword3, keyword4

Abstract

Abstracts should be 250 words. It must be able to stand alone and so cannot contain citations to the paper’s references, equations, etc. An abstract must consist of a single paragraph and be concise. Because of online formatting, abstracts must appear as plain as possible.

Contents

1	Technical Indicators and Feature Engineering - Data Layer	1
1.1	Mathematical Formulations of Basic Technical Indicators	2
1.1.1	Moving Average (MA)	2
1.1.2	Exponential Moving Average (EMA)	2
1.1.3	Moving Average Convergence Divergence (MACD)	2
1.1.4	Relative Strength Index (RSI)	2
1.1.5	Stochastic Oscillator (%K)	3
1.2	Feature Optimization and Tensor Transformation	3
2	Individual Architectures	3
2.1	Deep Sequential Baselines: Recurrent Networks (LSTM/BiLSTM)	3
2.1.1	Tensor Data Pipeline	3
2.1.2	Long-Short Term Memory Model (LSTM)	4
2.1.3	Bi-directional Long Short-Term Memory Model(BiLSTM)	5
2.2	Structural Signal Decomposition: N-BEATS	6
2.3	Tabular Master: LightGBM	6
3	Hybrid Architectures - The Integration Layer	6
4	The Large Language Model Shift	6
5	Benchmark	6
6	Conclusion	6

1. Technical Indicators and Feature Engineering - Data Layer

In financial time-series forecasting, raw market data is often characterized by a low signal-to-noise ratio, non-stationarity, and extreme volatility. To address these challenges and enhance the predictive accuracy of machine learning architectures, data preprocessing and feature engineering serve as a critical foundational layer. A widely adopted framework involves calculating Technical Indicators from raw market prices to transform sequential price paths into highly informative structural signals.

Therefore, we will introduce some basic technical indicators definition before introducing machine learning architectures.

1.1. Mathematical Formulations of Basic Technical Indicators

To systematically capture different dimensions of market dynamics, such as trend changes, price momentum, and overbought/oversold conditions, several classical statistical indicators are computed using historical open, high, low, and close (OHLC) prices

1.1.1. Moving Average (MA)

Moving Average helps smooth out price data to create a trend-following indicator, making it easier to identify price trends. Which can be designed in different lengths of windows, such as MA₅, MA₁₀, MA₂₀ etc.

$$MA_T = \frac{1}{T} \sum_{i=0}^T C_i \quad (1.1)$$

Where

- T : The length of windows.
- C_i : The closing price of day i

1.1.2. Exponential Moving Average (EMA)

For an Exponential Moving Average, the calculation gives more weight to more recent prices and is defined as.

$$EMA_T = \alpha C_i + (1 - \alpha)EMA_{T-1} \quad (1.2)$$

where

$$\alpha = \frac{2}{T + 1} \quad (1.3)$$

- T : The length of windows.
- C_i : The closing price of day i

1.1.3. Moving Average Convergence Divergence (MACD)

The Moving Average Convergence Divergence is a momentum-based technical indicator that calculates the difference between a short window EMA and a long window EMA. Usually using the difference between EMA₁₂ and EMA₂₆

$$MACD = EMA_{\text{long}} - EMA_{\text{short}} \quad (1.4)$$

1.1.4. Relative Strength Index (RSI)

The Relative Strength Index is a momentum oscillator that measures the speed and magnitude of recent price movements to evaluate overbought or oversold conditions in a market. Usually using RSI₆, RSI₁₂ and RSI₁₄.

$$RSI_T = 100 - \left[\frac{100}{1 + \left(\frac{AG_T}{AL_T} \right)} \right] \quad (1.5)$$

Where

- AG_T : The Average Gain in the over T days lookback window.
- AL_T : The Average Loss in the over T days lookback window.

1.1.5. Stochastic Oscillator (%K)

The Stochastic Oscillator is also momentum indicator comparing a specific closing price of a security to its price range over a given period T :

$$\%K = \left[\frac{C_i - L_T}{H_T - L_T} \right] \times 100 \quad (1.6)$$

Where

- C_i : The current closing price.
- L_T : The Lowest prices within the last T days.
- H_T : The Highest prices within the last T days.

1.2. Feature Optimization and Tensor Transformation

Upon generating the comprehensive pool of Technical Indicators, a crucial optimization step is required before model training. Raw indicators cannot be fed blindly into deep neural networks, as highly collinear features (such as MA, EMA, and MACD) or irrelevant features can lead to overfitting and severe computational inefficiency.

To extract the most predictive inputs, Pearson correlation (IC) analysis is performed to calculate the linear relationship between each individual technical indicator and the subsequent closing price. Features demonstrating the strongest historical correlation are isolated to filter out background market noise. These highly optimized adaptive features are then structured and stacked into a final multidimensional technical tensor.

2. Individual Architectures

2.1. Deep Sequential Baselines: Recurrent Networks (LSTM/BiLSTM)

Traditional time-series models like RNN often struggle to capture long-term temporal dependencies due to vanishing gradient problems in standard backpropagation. Long Short-Term Memory (LSTM) networks and their advanced variant, Bidirectional LSTMs (BiLSTM), resolve this by utilizing a specialized gating mechanism (comprising input, forget, and output gates) to retain information over long horizons.

While a standard LSTM processes inputs chronologically, a BiLSTM duplicates the recurrent layer to sweep data in both forward and backward time directions. This dual-directional approach allows the network to evaluate how past trajectories and future context concurrently impact current sequence steps, offering a robust baseline for dense sequential encoding.

2.1.1. Tensor Data Pipeline

Before feeding data into recurrent architectures, the optimized technical indicators and market variables processed in Chapter 1 must be strictly formatted into a three-dimensional tensor with the shape (Batch-Size, Time-Steps, Input-Features).

- Batch-Size: The number of training samples (e.g., historical trading sequences) processed simultaneously in a single iteration to stabilize gradient descent.

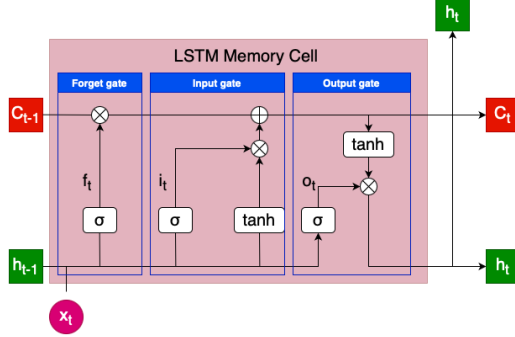


Figure 1. LSTM Workflow Diagram.

- Time-Steps (T): The length of the lookback historical window (e.g., using the past 30 days of market data to predict future movements).
- Input-Features (d): The total number of parallel channels, representing the raw market metrics (OHLCV) combined with the correlation-filtered Technical Indicators such as RSI and MACD.

When this 3-D tensor enters the recurrent network, the model does not look at the entire matrix at once. Instead, it sequentially unpacks the tensor along the Time-Steps axis. At each specific time step t (where $1 \leq t \leq T$), the network ingests a 1D feature vector $x_t \in \mathbb{R}^d$, representing the exact market snapshot of that specific moment. The mathematical mission of the recurrent cell is to map this continuous stream of vector inputs into an internal trajectory of memory states.

2.1.2. Long-Short Term Memory Model (LSTM)

Traditional Recurrent Neural Networks (RNNs) suffer from vanishing and exploding gradients during backpropagation through time, making it mathematically impossible to retain historical dependencies over long windows. To mitigate this constraint, the Long Short-Term Memory (LSTM) network introduces a specialized architectural cell governed by an explicit hidden state (h_t) and a cell state (c_t), which acts as a dedicated internal conveyor belt for long-term memory. At each time step t , the LSTM cell ingests the current feature input x_t and the previous step's hidden state h_{t-1} . The architecture of the LSTM cell is illustrated in Figure 1. And the information flow is dynamically regulated by three learnable, non-linear gates:

1. Forget Gate (f_t): Determines what percentage of the long-term memory (c_{t-1}) should be discarded.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.1)$$

where

- f_t : The output value of the Forget Gate, which has the range of 0 to 1.
 - W_f : The weight function of Forget Gate.
 - h_{t-1} : The output value of last time.
 - x_t : The current input value.
 - b_f : The bias of Forget Gate.
 - σ : The sigmoid layer which squashes the values between 0 to 1.
2. Input Gate (i_t & $\tilde{c}_t$): Decides which new market insights from x_t should be written into the long-term cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.2)$$

Where

- i_t : The output value of the Input Gate, which has the range of 0 to 1.
- W_i : The weight function of Input Gate.
- h_{t-1} : The output value of last time.
- x_t : The current input value.
- b_i : The bias of Input Gate.
- σ : The sigmoid layer which squashes the values between 0 to 1.

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (2.3)$$

Where

- $\tilde{c}_t$: The candidate cell state value.
- W_c : The weight function of candidate cell state.
- h_{t-1} : The output value of last time.
- x_t : The current input value.
- b_c : The bias of candidate cell state.
- $\tanh$: The hyperbolic tangent function squashes the values between -1 to 1.

Then, Cell State Update (c_t) Combines the filtered past memory and the validated new insights to form the updated long-term context.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (2.4)$$

3. Output Gate (o_t & h_t): Generates the current hidden state h_t , which serves as both the explicit model output for step t and the short-term contextual memory passed to step $t + 1$.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.5)$$

Where

- o_t : The output value of the Output Gate, which has the range of 0 to 1.
- W_o : The weight function of Output Gate.
- h_{t-1} : The output value of last time.
- x_t : The current input value.
- b_o : The bias of Output Gate.
- σ : The sigmoid layer which squashes the values between 0 to 1.

Then, update the current hidden state h_t

$$h_t = o_t \odot \tanh(c_t) \quad (2.6)$$

2.1.3. Bi-directional Long Short-Term Memory Model(BiLSTM)

In financial markets, price movements at time t are heavily influenced by local micro-trends and structural context that span across the entire temporal window. While a standard LSTM can only model information chronologically from left to right, the Bidirectional LSTM (BiLSTM) framework duplicates the internal hidden processing to scan the time series from both ends simultaneously.

2.2. Structural Signal Decomposition: N-BEATS

2.3. Tabular Master: LightGBM

3. Hybrid Architectures - The Integration Layer

4. The Large Language Model Shift

5. Benchmark

6. Conclusion